```
<!DOCTYPE html>
<html lang="en" class="dark">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>High-Dimensional Spatial Relationships & UMAP Geometry</title>
    <!-- Tailwind CSS -->
    <script src="https://cdn.tailwindcss.com"></script>
    <script>
        tailwind.config = {
            darkMode: 'class',
            theme: {
                extend: {
                    colors: {
                        brand: {
                            50: '#f0f7ff',
                            100: '#e0effe',
                            500: '#3b82f6',
                            600: '#2563eb',
                            700: '#1d4ed8',
                            900: '#1e3a8a',
                        },
                        hyperbolic: {
                            500: '#ec4899',
                            600: '#db2777',
                            700: '#be185d'
                        }
                    },
                    fontFamily: {
                        sans: ['Inter', 'sans-serif'],
                        mono: ['JetBrains Mono', 'Fira Code', 'monospace']
                    }
                }
            }
        }
    </script>
    <!-- MathJax for rendering LaTeX equations -->
    <script>
        window.MathJax = {
            tex: {
                inlineMath: [['', ''], ['\\(', '\\)']],
                displayMath: [['
                                                                ', '
'], ['\\[', '\\]']]
```

```
            },
            svg: {
                fontCache: 'global'
            }
        };
    </script>
    <script id="MathJax-script" async
src="https://cdn.jsdelivr.net/npm/mathjax@3/es5/tex-mml-chtml.js"></script>
    <!-- FontAwesome Icons -->
    <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
    <style>
        @import
url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&family=JetBrain
s+Mono:wght@400;500;700&display=swap');

        /* Custom scrollbars */
        ::-webkit-scrollbar {
            width: 6px;
            height: 6px;
        }
        ::-webkit-scrollbar-track {
            background: rgba(15, 23, 42, 0.6);
        }
        ::-webkit-scrollbar-thumb {
            background: rgba(100, 116, 139, 0.5);
            border-radius: 4px;
        }
        ::-webkit-scrollbar-thumb:hover {
            background: rgba(148, 163, 184, 0.8);
        }

        .glass-panel {
            background: rgba(30, 41, 59, 0.7);
            backdrop-filter: blur(12px);
            border: 1px solid rgba(255, 255, 255, 0.08);
        }
        .light .glass-panel {
            background: rgba(255, 255, 255, 0.85);
            backdrop-filter: blur(12px);
            border: 1px solid rgba(0, 0, 0, 0.08);
        }

        .tab-btn.active {
```

```
            border-bottom: 2px solid #3b82f6;
            color: #3b82f6;
        }
    </style>
</head>
<body class="bg-slate-950 text-slate-100 font-sans min-h-screen flex flex-col transition-colors
duration-200 light:bg-slate-50 light:text-slate-900">

    <!-- Header Navigation -->
    <header class="sticky top-0 z-50 glass-panel border-b border-slate-800
light:border-slate-200">
        <div class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 h-16 flex items-center
justify-between">
            <div class="flex items-center space-x-3">
                <div class="p-2 bg-brand-600 rounded-lg text-white font-bold text-xl shadow-lg
shadow-brand-500/30">
                    <i class="fa-solid font-mono">$R^N$</i>
                </div>
                <div>
                    <h1 class="font-bold text-lg leading-tight tracking-tight text-white
light:text-slate-900">
                        High-Dim Geometry & UMAP
                    </h1>
                    <p class="text-xs text-slate-400 light:text-slate-500">Spatial Relationships &
Embedding Metrics</p>
                </div>
            </div>

            <!-- Navigation Tabs -->
            <nav class="hidden md:flex space-x-6 text-sm font-medium">
                <button onclick="switchTab('calculator')" id="nav-calculator" class="tab-btn active py-5
px-1 text-slate-300 hover:text-white transition">
                    <i class="fa-solid fa-calculator mr-2"></i>Vector Calculator
                </button>
                <button onclick="switchTab('graph')" id="nav-graph" class="tab-btn py-5 px-1
text-slate-400 hover:text-white transition">
                    <i class="fa-solid fa-cube mr-2"></i>3D Visual Measurement
                </button>
                <button onclick="switchTab('umap')" id="nav-umap" class="tab-btn py-5 px-1
text-slate-400 hover:text-white transition">
                    <i class="fa-solid fa-network-wired mr-2"></i>UMAP & Hyperbolic Visualizer
                </button>
                <button onclick="switchTab('formulas')" id="nav-formulas" class="tab-btn py-5 px-1
text-slate-400 hover:text-white transition">
```

```
                <i class="fa-solid fa-square-root-variable mr-2"></i>Math Foundations
            </button>
            <button onclick="switchTab('glossary')" id="nav-glossary" class="tab-btn py-5 px-1
text-slate-400 hover:text-white transition">
                <i class="fa-solid fa-book-bookmark mr-2"></i>Glossary
            </button>
        </nav>

        <!-- Actions -->
        <div class="flex items-center space-x-3">
            <button id="themeToggle" onclick="toggleTheme()" class="p-2 rounded-lg
bg-slate-800 text-slate-300 hover:text-white hover:bg-slate-700 light:bg-slate-200
light:text-slate-700 transition" title="Toggle Light/Dark Theme">
                <i class="fa-solid fa-moon text-lg" id="themeIcon"></i>
            </button>
        </div>
    </div>

    <!-- Mobile Navigation -->
    <div class="md:hidden flex border-t border-slate-800 light:border-slate-200 px-2 py-2
overflow-x-auto space-x-2 text-xs">
        <button onclick="switchTab('calculator')" id="mob-nav-calculator" class="px-3 py-1.5
rounded-md bg-brand-600 text-white font-medium whitespace-nowrap">Calculator</button>
        <button onclick="switchTab('graph')" id="mob-nav-graph" class="px-3 py-1.5
rounded-md text-slate-400 hover:bg-slate-800 whitespace-nowrap">3D Graph</button>
        <button onclick="switchTab('umap')" id="mob-nav-umap" class="px-3 py-1.5 rounded-md
text-slate-400 hover:bg-slate-800 whitespace-nowrap">UMAP Visualizer</button>
        <button onclick="switchTab('formulas')" id="mob-nav-formulas" class="px-3 py-1.5
rounded-md text-slate-400 hover:bg-slate-800 whitespace-nowrap">Math</button>
        <button onclick="switchTab('glossary')" id="mob-nav-glossary" class="px-3 py-1.5
rounded-md text-slate-400 hover:bg-slate-800 whitespace-nowrap">Glossary</button>
    </div>
</header>

<!-- Main Body Container -->
<main class="flex-grow max-w-7xl w-full mx-auto px-4 sm:px-6 lg:px-8 py-6 space-y-6">

    <!-- ================= TAB 1: VECTOR CALCULATOR ================= -->
    <section id="tab-calculator" class="space-y-6">
        <div class="grid grid-cols-1 lg:grid-cols-12 gap-6">
            <!-- Left Input Controls (5 cols) -->
            <div class="lg:col-span-5 space-y-4">
                <div class="glass-panel p-5 rounded-2xl border shadow-xl">
                    <div class="flex justify-between items-center mb-3">
```

```
                    <h2 class="text-base font-semibold flex items-center text-white
light:text-slate-900">
                        <i class="fa-solid fa-sliders text-brand-500 mr-2"></i> Vector Inputs
                    </h2>
                    <div class="space-x-1">
                        <button onclick="generateRandomVectors(3)" class="text-xs px-2.5 py-1
rounded bg-slate-800 hover:bg-slate-700 text-slate-300 light:bg-slate-200 light:text-slate-800
transition">3D</button>
                        <button onclick="generateRandomVectors(128)" class="text-xs px-2.5 py-1
rounded bg-slate-800 hover:bg-slate-700 text-slate-300 light:bg-slate-200 light:text-slate-800
transition">128D</button>
                        <button onclick="generateRandomVectors(768)" class="text-xs px-2.5 py-1
rounded bg-brand-600/20 text-brand-400 hover:bg-brand-600/30 border border-brand-500/30
font-semibold transition">768D (LLM)</button>
                    </div>
                </div>

                <!-- Error Banner -->
                <div id="calcError" class="hidden mb-4 p-3 rounded-xl bg-red-950/60 border
border-red-500/50 text-red-300 text-xs flex items-start space-x-2">
                    <i class="fa-solid fa-triangle-exclamation text-red-400 mt-0.5"></i>
                    <span id="calcErrorMsg">Dimensions do not match.</span>
                </div>

                <!-- Input Vector A -->
                <div class="space-y-1.5 mb-4">
                    <div class="flex justify-between items-center text-xs">
                        <label class="font-medium text-slate-300 light:text-slate-700 flex
items-center">
                            <span class="w-2.5 h-2.5 rounded-full bg-brand-500 inline-block
mr-1.5"></span> Vector A ($u$)
                        </label>
                        <span id="dimBadgeA" class="font-mono text-slate-500 text-[10px]">Dim:
0</span>
                    </div>
                    <textarea id="vectorAInput" rows="4" class="w-full bg-slate-900/90
light:bg-white border border-slate-700 light:border-slate-300 rounded-xl p-3 text-xs font-mono
text-slate-200 light:text-slate-800 focus:ring-2 focus:ring-brand-500 focus:outline-none
transition" placeholder="e.g. 0.25, -0.74, 0.81, 0.12..."></textarea>
                </div>

                <!-- Input Vector B -->
                <div class="space-y-1.5 mb-4">
                    <div class="flex justify-between items-center text-xs">
```

```
                        <label class="font-medium text-slate-300 light:text-slate-700 flex
items-center">
                            <span class="w-2.5 h-2.5 rounded-full bg-emerald-500 inline-block
mr-1.5"></span> Vector B ($v$)
                        </label>
                        <span id="dimBadgeB" class="font-mono text-slate-500 text-[10px]">Dim:
0</span>
                    </div>
                    <textarea id="vectorBInput" rows="4" class="w-full bg-slate-900/90
light:bg-white border border-slate-700 light:border-slate-300 rounded-xl p-3 text-xs font-mono
text-slate-200 light:text-slate-800 focus:ring-2 focus:ring-brand-500 focus:outline-none
transition" placeholder="e.g. 0.18, -0.62, 0.95, 0.04..."></textarea>
                </div>

                <!-- Action Controls -->
                <div class="flex space-x-2">
                    <button onclick="runCalculation()" class="flex-1 bg-brand-600
hover:bg-brand-500 text-white font-medium py-2.5 px-4 rounded-xl text-xs flex items-center
justify-center space-x-2 shadow-lg shadow-brand-600/25 transition">
                        <i class="fa-solid fa-play"></i>
                        <span>Compute Metrics</span>
                    </button>
                    <button onclick="normalizeInputs()" class="bg-slate-800 hover:bg-slate-700
light:bg-slate-200 light:hover:bg-slate-300 text-slate-300 light:text-slate-700 font-medium py-2.5
px-3 rounded-xl text-xs transition" title="L2 Normalize Vectors to Unit Length">
                        $\hat{u}, \hat{v}$ L2 Norm
                    </button>
                </div>
            </div>

            <!-- Input Vector Properties Summary -->
            <div class="glass-panel p-4 rounded-2xl border text-xs space-y-2">
                <h3 class="font-semibold text-slate-400 light:text-slate-600 uppercase
tracking-wider text-[11px]">Vector Magnitudes ($L_2$ Norm)</h3>
                <div class="grid grid-cols-2 gap-3 font-mono">
                    <div class="bg-slate-900/50 light:bg-slate-100 p-2.5 rounded-xl border
border-slate-800 light:border-slate-200">
                        <div class="text-slate-400 text-[10px]">$\|\mathbf{u}\|_2$</div>
                        <div id="normValA" class="text-sm font-semibold
text-brand-400">0.0000</div>
                    </div>
                    <div class="bg-slate-900/50 light:bg-slate-100 p-2.5 rounded-xl border
border-slate-800 light:border-slate-200">
```

```
                        <div class="text-slate-400 text-[10px]">$\|\mathbf{v}\|_2$</div>
                        <div id="normValB" class="text-sm font-semibold
text-emerald-400">0.0000</div>
                    </div>
                </div>
            </div>

            <!-- Right Results Dashboard (7 cols) -->
            <div class="lg:col-span-7 space-y-4">
                <!-- Metric Display Grid -->
                <div class="grid grid-cols-2 sm:grid-cols-3 gap-3">
                    <!-- Cosine Similarity -->
                    <div class="glass-panel p-4 rounded-2xl border relative overflow-hidden group">
                        <div class="text-xs text-slate-400 font-medium">Cosine Similarity</div>
                        <div id="resCosineSim" class="text-xl sm:text-2xl font-bold font-mono
text-white light:text-slate-900 my-1">--</div>
                        <div class="text-[10px] text-slate-500 font-mono">Range: [-1.0, 1.0]</div>
                        <div class="absolute bottom-0 left-0 right-0 h-1 bg-slate-800">
                            <div id="barCosine" class="h-full bg-brand-500 transition-all duration-300"
style="width: 0%"></div>
                        </div>
                    </div>

                    <!-- Euclidean Distance -->
                    <div class="glass-panel p-4 rounded-2xl border relative overflow-hidden">
                        <div class="text-xs text-slate-400 font-medium">Euclidean ($L_2$)</div>
                        <div id="resEuclidean" class="text-xl sm:text-2xl font-bold font-mono text-white
light:text-slate-900 my-1">--</div>
                        <div class="text-[10px] text-slate-500 font-mono">Straight Line $d_2$</div>
                        <div class="absolute bottom-0 left-0 right-0 h-1 bg-slate-800">
                            <div id="barEuclidean" class="h-full bg-indigo-500 transition-all
duration-300" style="width: 0%"></div>
                        </div>
                    </div>

                    <!-- Cosine Distance -->
                    <div class="glass-panel p-4 rounded-2xl border relative overflow-hidden">
                        <div class="text-xs text-slate-400 font-medium">Cosine Distance</div>
                        <div id="resCosDist" class="text-xl sm:text-2xl font-bold font-mono text-white
light:text-slate-900 my-1">--</div>
                        <div class="text-[10px] text-slate-500 font-mono">$1 - Sim_{\cos}$</div>
                        <div class="absolute bottom-0 left-0 right-0 h-1 bg-slate-800">
```

```
                        <div id="barCosDist" class="h-full bg-purple-500 transition-all duration-300"
style="width: 0%"></div>
                    </div>
                </div>

                <!-- Manhattan Distance -->
                <div class="glass-panel p-4 rounded-2xl border relative overflow-hidden">
                    <div class="text-xs text-slate-400 font-medium">Manhattan ($L_1$)</div>
                    <div id="resManhattan" class="text-xl sm:text-2xl font-bold font-mono
text-white light:text-slate-900 my-1">--</div>
                    <div class="text-[10px] text-slate-500 font-mono">City Block Sum</div>
                </div>

                <!-- Dot Product -->
                <div class="glass-panel p-4 rounded-2xl border relative overflow-hidden">
                    <div class="text-xs text-slate-400 font-medium">Dot Product</div>
                    <div id="resDot" class="text-xl sm:text-2xl font-bold font-mono text-white
light:text-slate-900 my-1">--</div>
                    <div class="text-[10px] text-slate-500 font-mono">$u \cdot v$</div>
                </div>

                <!-- Angular Distance -->
                <div class="glass-panel p-4 rounded-2xl border relative overflow-hidden">
                    <div class="text-xs text-slate-400 font-medium">Angular Offset</div>
                    <div id="resAngle" class="text-xl sm:text-2xl font-bold font-mono
text-amber-400 my-1">--°</div>
                    <div class="text-[10px] text-slate-500 font-mono">$\theta = \arccos(Sim)$</div>
                </div>
            </div>

            <!-- Numerical & Arithmetic Breakdown Accordion -->
            <div class="glass-panel p-5 rounded-2xl border space-y-3">
                <div class="flex justify-between items-center">
                    <h3 class="font-semibold text-sm text-slate-200 light:text-slate-800 flex
items-center">
                        <i class="fa-solid fa-square-poll-vertical text-brand-500 mr-2"></i>
Single-Pass Compute Breakdown
                    </h3>
                    <button onclick="copyResults()" class="text-xs px-2.5 py-1 rounded
bg-slate-800 hover:bg-slate-700 text-slate-300 light:bg-slate-200 light:text-slate-700 transition">
                        <i class="fa-regular fa-copy mr-1"></i> Copy JSON
                    </button>
                </div>
```

// Single-pass $O(N)$ Accumulation Steps:

1. Dot Product ($\sum u_i v_i$) = 0
2. $\|\mathbf{u}\|^2_2 = \sum u_i^2$ = 0 → $\|\mathbf{u}\| = 0$
3. $\|\mathbf{v}\|^2_2 = \sum v_i^2$ = 0 → $\|\mathbf{v}\| = 0$
4. Cosine Sim = $\frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$ = 0

5. Euclidean = $\sqrt{\sum(u_i - v_i)^2}$ = 0

<!-- High Dimensions Insight Card -->

 High-Dimensional Geometry Phenomenon (Curse of Dimensionality)

In high-dimensional embedding spaces (e.g., $N = 768$ or 1536), almost all random vectors become nearly orthogonal ($Sim_{\cos} \approx 0$). Furthermore, Euclidean distance contrast deteriorates as $\frac{D_{\max} - D_{\min}}{D_{\min}} \to 0$, making angle-based metrics (Cosine Similarity) preferred for vector databases and language model embeddings.

</section>

<!-- ================= TAB 2: 3D VISUAL MEASUREMENT GRAPH ================= -->
<section id="tab-graph" class="hidden space-y-6">

```
        <div class="glass-panel p-6 rounded-2xl border shadow-xl">
            <div class="flex flex-col md:flex-row md:items-center justify-between pb-4 border-b
border-slate-800 light:border-slate-200 gap-4">
                <div>
                    <h2 class="text-lg font-bold text-white light:text-slate-900 flex items-center">
                        <i class="fa-solid fa-cube text-brand-500 mr-2.5"></i> Interactive 3D
High-Dimensional Spatial Mapping & Coordinate Plotter
                    </h2>
                    <p class="text-xs text-slate-400 light:text-slate-600 mt-1">
                        Collapse arbitrary high-dimensional subsets into composite 3D axes (X, Y, Z),
apply radial distance transformations, and orbit in 3D space.
                    </p>
                </div>
                <div class="flex flex-wrap gap-2">
                    <button onclick="syncGraphFromCalculator()" class="text-xs px-3 py-1.5
rounded-lg bg-brand-600 hover:bg-brand-500 text-white font-medium shadow transition flex
items-center">
                        <i class="fa-solid fa-arrows-rotate mr-1.5"></i> Sync from Vector Inputs
                    </button>
                    <button onclick="reset3DCamera()" class="text-xs px-3 py-1.5 rounded-lg
bg-slate-800 hover:bg-slate-700 text-slate-300 light:bg-slate-200 light:text-slate-800
font-medium transition">
                        <i class="fa-solid fa-camera mr-1"></i> Reset View
                    </button>
                </div>
            </div>

            <div class="grid grid-cols-1 lg:grid-cols-12 gap-6 mt-6">
                <!-- Controls Column -->
                <div class="lg:col-span-5 space-y-4">
                    <!-- High-Dim Plane & Dimension Collapsing Controller -->
                    <div class="glass-panel p-4 rounded-xl border space-y-3 bg-brand-950/20
border-brand-500/20">
                        <div class="flex justify-between items-center text-xs">
                            <span class="font-semibold text-brand-400 light:text-brand-700 flex
items-center">
                                <i class="fa-solid fa-layer-group mr-1.5"></i> High-Dim Projection &
Collapsing
                            </span>
                            <span id="highDimBadge" class="font-mono text-[10px]
text-slate-400">Dim: 6D</span>
                        </div>

                        <!-- X Axis Mapping Selection -->
```

```
                    <div class="space-y-1 text-xs">
                        <label class="text-[11px] font-medium text-slate-300 light:text-slate-700 flex
justify-between">
                            <span>3D Axis X Mapping</span>
                            <span class="text-slate-500 text-[10px]">Basis Coordinate 1</span>
                        </label>
                        <select id="selectXDim" onchange="updateDimensionMapping()"
class="w-full bg-slate-900 light:bg-white border border-slate-700 light:border-slate-300
rounded-lg p-1.5 text-xs text-slate-200 light:text-slate-800 focus:ring-2 focus:ring-brand-500
outline-none">
                            <!-- Options generated dynamically -->
                        </select>
                    </div>

                    <!-- Y Axis Mapping Selection -->
                    <div class="space-y-1 text-xs">
                        <label class="text-[11px] font-medium text-slate-300 light:text-slate-700 flex
justify-between">
                            <span>3D Axis Y Mapping</span>
                            <span class="text-slate-500 text-[10px]">Basis Coordinate 2</span>
                        </label>
                        <select id="selectYDim" onchange="updateDimensionMapping()"
class="w-full bg-slate-900 light:bg-white border border-slate-700 light:border-slate-300
rounded-lg p-1.5 text-xs text-slate-200 light:text-slate-800 focus:ring-2 focus:ring-brand-500
outline-none">
                            <!-- Options generated dynamically -->
                        </select>
                    </div>

                    <!-- Z Axis / Collapsed Dimensions Selection -->
                    <div class="space-y-1 text-xs">
                        <label class="text-[11px] font-medium text-slate-300 light:text-slate-700 flex
justify-between">
                            <span>3D Axis Z / Radial Mapping</span>
                            <span class="text-slate-500 text-[10px]">Basis Coordinate 3 /
Radius</span>
                        </label>
                        <select id="selectZDim" onchange="updateDimensionMapping()"
class="w-full bg-slate-900 light:bg-white border border-slate-700 light:border-slate-300
rounded-lg p-1.5 text-xs text-slate-200 light:text-slate-800 focus:ring-2 focus:ring-brand-500
outline-none">
                            <!-- Options generated dynamically -->
                        </select>
                    </div>
```

```
                    <!-- Dimension Collapsing Mode -->
                    <div class="pt-2 border-t border-slate-800 light:border-slate-200 grid
grid-cols-2 gap-2 text-xs">
                        <div>
                            <label class="text-[10px] text-slate-400 block mb-0.5">Collapse Subset
$d_1 \dots d_k$</label>
                            <select id="selectCollapseMode"
onchange="updateDimensionMapping()" class="w-full bg-slate-900 light:bg-white border
border-slate-700 light:border-slate-300 rounded-lg p-1.5 text-xs text-slate-200
light:text-slate-800">
                                <option value="none">Single Dimension Direct</option>
                                <option value="l2norm" selected>Collapse Subset ($L_2$ Norm)</option>
                                <option value="mean">Collapse Subset (Mean)</option>
                                <option value="sum">Collapse Subset (Sum)</option>
                            </select>
                        </div>
                        <div>
                            <label class="text-[10px] text-slate-400 block mb-0.5">Radial Distance
Constraint</label>
                            <select id="selectRadialMode" onchange="updateDimensionMapping()"
class="w-full bg-slate-900 light:bg-white border border-slate-700 light:border-slate-300
rounded-lg p-1.5 text-xs text-slate-200 light:text-slate-800">
                                <option value="off">Off (Cartesian 3D)</option>
                                <option value="dim4" selected>Use $d_4$ as Radius from Origin</option>
                                <option value="fullNorm">Use Full Vector Magnitude
$\|\mathbf{v}\|_2$</option>
                            </select>
                        </div>
                    </div>
                </div>

                <!-- 3D Overlays & Controls -->
                <div class="glass-panel p-4 rounded-xl border space-y-3">
                    <h3 class="text-xs font-semibold text-slate-300 light:text-slate-700 uppercase
tracking-wider">
                        3D Overlay Visuals
                    </h3>
                    <div class="grid grid-cols-2 gap-2 text-xs">
                        <label class="flex items-center space-x-2 text-slate-200 light:text-slate-800
cursor-pointer">
                            <input type="checkbox" id="chkEuclidean" checked
onchange="renderGraph()" class="rounded accent-indigo-500">
```

```
                        <span class="w-2.5 h-0.5 bg-indigo-500 inline-block"></span>
                        <span>3D Euclidean Line</span>
                    </label>
                    <label class="flex items-center space-x-2 text-slate-200 light:text-slate-800
cursor-pointer">
                        <input type="checkbox" id="chkManhattan" checked
onchange="renderGraph()" class="rounded accent-purple-500">
                        <span class="w-2.5 h-0.5 border-b border-dashed border-purple-500
inline-block"></span>
                        <span>3D Taxicab Path</span>
                    </label>
                    <label class="flex items-center space-x-2 text-slate-200 light:text-slate-800
cursor-pointer">
                        <input type="checkbox" id="chkAngleArc" checked
onchange="renderGraph()" class="rounded accent-amber-500">
                        <span class="w-2.5 h-2.5 rounded-full border border-amber-500
inline-block"></span>
                        <span>3D Angle Arc (θ)</span>
                    </label>
                    <label class="flex items-center space-x-2 text-slate-200 light:text-slate-800
cursor-pointer">
                        <input type="checkbox" id="chkVectorProj" checked
onchange="renderGraph()" class="rounded accent-emerald-500">
                        <span class="w-2.5 h-0.5 border-b border-dotted border-emerald-400
inline-block"></span>
                        <span>Vector Projection</span>
                    </label>
                    <label class="flex items-center space-x-2 text-slate-200 light:text-slate-800
cursor-pointer">
                        <input type="checkbox" id="chkProjection" checked
onchange="renderGraph()" class="rounded accent-cyan-500">
                        <span class="w-2.5 h-0.5 border-b border-dotted border-cyan-400
inline-block"></span>
                        <span>Floor Drop Lines</span>
                    </label>
                    <label class="flex items-center space-x-2 text-slate-200 light:text-slate-800
cursor-pointer">
                        <input type="checkbox" id="chkBoundingBox" checked
onchange="renderGraph()" class="rounded accent-amber-500">
                        <span class="w-2 h-2 rounded-sm bg-amber-500/30
inline-block"></span>
                        <span>3D Wire Volume</span>
                    </label>
                </div>
```

```
            </div>

            <!-- Real-Time Measured Metrics Panel -->
            <div class="glass-panel p-4 rounded-xl border space-y-2.5 text-xs">
              <h3 class="font-semibold text-slate-400 light:text-slate-600 uppercase
tracking-wider text-[11px] flex justify-between">
                <span>3D Projected Coordinates</span>
                <span class="text-brand-400 font-mono">Real-Time</span>
              </h3>
              <div class="space-y-1.5 font-mono">
                <div class="flex justify-between p-2 rounded bg-slate-900/60
light:bg-slate-100">
                  <span class="text-slate-400">Vector A Projected (X, Y, Z):</span>
                  <span id="graphCoordA" class="text-brand-400 font-bold">[0.0, 0.0,
0.0]</span>
                </div>
                <div class="flex justify-between p-2 rounded bg-slate-900/60
light:bg-slate-100">
                  <span class="text-slate-400">Vector B Projected (X, Y, Z):</span>
                  <span id="graphCoordB" class="text-emerald-400 font-bold">[0.0, 0.0,
0.0]</span>
                </div>
                <div class="flex justify-between p-2 rounded bg-slate-900/60
light:bg-slate-100">
                  <span class="text-slate-400">3D Projected Euclidean:</span>
                  <span id="graphValEuclid" class="text-indigo-400
font-bold">0.000</span>
                </div>
                <div class="flex justify-between p-2 rounded bg-slate-900/60
light:bg-slate-100">
                  <span class="text-slate-400">3D Projected Manhattan:</span>
                  <span id="graphValManhattan" class="text-purple-400
font-bold">0.000</span>
                </div>
                <div class="flex justify-between p-2 rounded bg-slate-900/60
light:bg-slate-100">
                  <span class="text-slate-400">Projected Angle (θ_3D):</span>
                  <span id="graphValAngle" class="text-amber-400 font-bold">0.0°</span>
                </div>
              </div>
            </div>

            <!-- Canvas Column with 3D Interaction -->
```

```
                <div class="lg:col-span-7 flex flex-col">
                    <div class="relative w-full aspect-square max-h-[520px] bg-slate-950 light:bg-white border border-slate-800 light:border-slate-300 rounded-2xl overflow-hidden shadow-inner cursor-grab active:cursor-grabbing">
                        <canvas id="graphCanvas" class="w-full h-full"></canvas>

                        <!-- Overlay Help -->
                        <div class="absolute bottom-3 left-3 pointer-events-none bg-slate-900/80 light:bg-white/90 backdrop-blur px-3 py-1.5 rounded-lg border border-slate-800 light:border-slate-200 text-[10px] font-mono text-slate-400 flex items-center space-x-2">
                            <i class="fa-solid fa-arrows-spin text-brand-400"></i>
                            <span>Click & Drag to Orbit Camera in 3D Space</span>
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </section>

    <!-- ================= TAB 3: UMAP & HYPERBOLIC VISUALIZER ================= -->
    <section id="tab-umap" class="hidden space-y-6">
        <div class="glass-panel p-6 rounded-2xl border shadow-xl">
            <div class="flex flex-col md:flex-row md:items-center justify-between pb-4 border-b border-slate-800 light:border-slate-200 gap-4">
                <div>
                    <h2 class="text-lg font-bold text-white light:text-slate-900 flex items-center">
                        <i class="fa-solid fa-network-wired text-brand-500 mr-2.5"></i> Interactive UMAP Metric & Poincaré Hyperbolic Visualizer
                    </h2>
                    <p class="text-xs text-slate-400 light:text-slate-600 mt-1">
                        Explore how high-dimensional input metrics map points into low-dimensional Euclidean planes versus Hyperbolic Poincaré Disks.
                    </p>
                </div>
                <div class="flex flex-wrap gap-2">
                    <button onclick="setUmapPreset('hierarchical')" class="text-xs px-3 py-1.5 rounded-lg bg-slate-800 hover:bg-slate-700 text-slate-300 light:bg-slate-200 light:text-slate-800 font-medium transition">
                        <i class="fa-solid fa-sitemap mr-1"></i> Tree Hierarchy
                    </button>
                    <button onclick="setUmapPreset('clusters')" class="text-xs px-3 py-1.5 rounded-lg bg-slate-800 hover:bg-slate-700 text-slate-300 light:bg-slate-200 light:text-slate-800 font-medium transition">
```

```
                    <i class="fa-solid fa-circle-nodes mr-1"></i> Clusters
                </button>
            </div>
        </div>

        <div class="grid grid-cols-1 lg:grid-cols-12 gap-6 mt-6">
            <!-- Controls Column -->
            <div class="lg:col-span-5 space-y-5">
                <div class="space-y-2">
                    <label class="text-xs font-semibold text-slate-300 light:text-slate-700 block
uppercase tracking-wider">
                        1. High-Dim Input Metric (<span class="text-brand-400">metric =</span>)
                    </label>
                    <select id="umapInputMetric" onchange="updateUmapSimulation()"
class="w-full bg-slate-900 light:bg-white border border-slate-700 light:border-slate-300
rounded-xl p-2.5 text-xs text-slate-200 light:text-slate-800 focus:ring-2 focus:ring-brand-500
outline-none">
                        <option value="euclidean">Euclidean (L2 distance)</option>
                        <option value="manhattan">Manhattan (L1 taxicab)</option>
                        <option value="cosine" selected>Cosine Similarity / Angle</option>
                    </select>
                </div>

                <div class="space-y-2">
                    <label class="text-xs font-semibold text-slate-300 light:text-slate-700 block
uppercase tracking-wider">
                        2. Low-Dim Output Metric (<span class="text-hyperbolic-500">
output_metric =</span>)
                    </label>
                    <select id="umapOutputMetric" onchange="updateUmapSimulation()"
class="w-full bg-slate-900 light:bg-white border border-slate-700 light:border-slate-300
rounded-xl p-2.5 text-xs text-slate-200 light:text-slate-800 focus:ring-2 focus:ring-brand-500
outline-none">
                        <option value="euclidean">Euclidean 2D Plane (R^2)</option>
                        <option value="hyperboloid" selected>Hyperboloid → Poincaré Disk (H^2
)</option>
                    </select>
                </div>

                <div class="space-y-4 pt-2 border-t border-slate-800 light:border-slate-200">
                    <div>
                        <div class="flex justify-between text-xs mb-1">
```

```
                        <span class="text-slate-300 light:text-slate-700">Nearest Neighbors ( n_neighbors)</span>
                        <span id="valNeighbors" class="font-mono text-brand-400">15</span>
                    </div>
                    <input type="range" id="sliderNeighbors" min="5" max="50" value="15" oninput="updateUmapSimulation()" class="w-full accent-brand-500">
                </div>

                <div>
                    <div class="flex justify-between text-xs mb-1">
                        <span class="text-slate-300 light:text-slate-700">Hyperbolic Curvature (κ )</span>
                        <span id="valCurvature" class="font-mono text-hyperbolic-500">1.0</span>
                    </div>
                    <input type="range" id="sliderCurvature" min="0.2" max="3.0" step="0.1" value="1.0" oninput="updateUmapSimulation()" class="w-full accent-hyperbolic-500">
                </div>
            </div>

            <!-- LIVE PYTHON CODE EXPORTER -->
            <div class="glass-panel p-4 rounded-xl border space-y-2">
                <div class="flex justify-between items-center">
                    <span class="text-[11px] font-semibold text-slate-300 light:text-slate-700 uppercase tracking-wider flex items-center">
                        <i class="fa-solid fa-code text-brand-500 mr-1.5"></i> Live Python Snippet
                    </span>
                    <button onclick="copyPythonCode()" class="text-[10px] px-2 py-0.5 rounded bg-slate-800 hover:bg-slate-700 text-slate-300 light:bg-slate-200 light:text-slate-700 transition">
                        <i class="fa-regular fa-copy mr-1"></i> Copy
                    </button>
                </div>
                <pre id="pythonCodeSnippet" class="bg-slate-950 light:bg-slate-100 p-3 rounded-lg font-mono text-[10px] text-brand-300 light:text-brand-700 overflow-x-auto border border-slate-800 light:border-slate-300 leading-relaxed"></pre>
            </div>
        </div>

        <!-- Canvas Visualization Area -->
        <div class="lg:col-span-7 flex flex-col">
            <div class="relative w-full aspect-square max-h-[520px] bg-slate-950 light:bg-white border border-slate-800 light:border-slate-300 rounded-2xl overflow-hidden shadow-inner flex items-center justify-center">
```

```
                    <canvas id="umapCanvas" class="w-full h-full cursor-crosshair"></canvas>

                    <div class="absolute top-3 left-3 pointer-events-none bg-slate-900/80
light:bg-white/90 backdrop-blur px-3 py-2 rounded-lg border border-slate-800
light:border-slate-200 text-[11px] font-mono space-y-1">
                        <div class="flex items-center space-x-2">
                            <span class="w-2.5 h-2.5 rounded-full bg-brand-500
inline-block"></span>
                            <span class="text-slate-300 light:text-slate-700">Cluster Nodes</span>
                        </div>
                        <div class="flex items-center space-x-2">
                            <span class="w-3 h-0.5 bg-brand-500/40 inline-block"></span>
                            <span class="text-slate-400">High-Dim Fuzzy Edge</span>
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </section>

    <!-- ================= TAB 4: MATHEMATICAL FOUNDATIONS
================= -->
    <section id="tab-formulas" class="hidden space-y-6">
        <div class="text-center max-w-3xl mx-auto space-y-2">
            <h2 class="text-2xl font-bold text-white light:text-slate-900">High-Dimensional
Mathematical Foundations</h2>
            <p class="text-sm text-slate-400 light:text-slate-600">
                Detailed equations, norm definitions, metric axioms, and manifold geometry
principles.
            </p>
        </div>

        <div class="grid grid-cols-1 md:grid-cols-2 gap-6">
            <!-- Euclidean Card -->
            <div class="glass-panel p-6 rounded-2xl border space-y-4">
                <div class="flex justify-between items-start">
                    <div>
                        <span class="text-[10px] font-mono uppercase tracking-widest
text-brand-400">L2 Norm Metric</span>
                        <h3 class="text-lg font-bold text-white light:text-slate-900">Euclidean
Distance</h3>
                    </div>
```

$\|u - v\|_2$

$$d_{Euclidean}(u, v) = \sqrt{\sum_{i=1}^{N} (u_i - v_i)^2}$$

<ul class="text-xs text-slate-300 light:text-slate-700 space-y-2 list-disc list-inside leading-relaxed">
Geometric Interpretation: Straight-line distance in flat Euclidean space R^N.
Sensitivity: Sensitive to absolute component differences and scaling.

<!-- Cosine Card -->

Angular Metric
<h3 class="text-lg font-bold text-white light:text-slate-900">Cosine Similarity & Distance</h3>

$\cos(\theta)$

$$\text{Sim}_{\cos}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} = \frac{\sum u_i v_i}{\sqrt{\sum u_i^2} \sqrt{\sum v_i^2}}$$

<ul class="text-xs text-slate-300 light:text-slate-700 space-y-2 list-disc list-inside leading-relaxed">

Geometric Interpretation: Cosine of the angle θ between vectors.
Invariance: Invariant to magnitude scaling, ideal for high-dim semantic embeddings.

<!-- Manhattan Card -->

L1 Norm Metric
<h3 class="text-lg font-bold text-white light:text-slate-900">Manhattan (Taxicab) Distance</h3>

$\|u - v\|_1$

$$d_{Manhattan}(u, v) = \sum_{i=1}^{N} |u_i - v_i|$$

<ul class="text-xs text-slate-300 light:text-slate-700 space-y-2 list-disc list-inside leading-relaxed">
Geometric Interpretation: Axis-aligned step distance along grid paths.
Robustness: Often preserves relative distance contrast better in high dimensions than L_2.

<!-- Mahalanobis Distance Card -->

Covariance-Weighted

<h3 class="text-lg font-bold text-white light:text-slate-900">Mahalanobis Distance</h3>

Σ^{-1}

$$d_{Mahalanobis}(u, v) = \sqrt{(u - v)^T \Sigma^{-1} (u - v)}$$

<ul class="text-xs text-slate-300 light:text-slate-700 space-y-2 list-disc list-inside leading-relaxed">
Covariance Scaling: Measures distance relative to feature correlations and variable variance (Σ).
Equivalence: Reduces to Euclidean distance when features are uncorrelated with unit variance ($\Sigma = I$).

<!-- Poincaré Hyperbolic Card -->

Non-Euclidean Space
<h3 class="text-lg font-bold text-white light:text-slate-900">Poincaré Hyperbolic Metric</h3>

H^N

$$d_{\mathcal{H}}(\mathbf{u}, \mathbf{v}) = \text{arcosh}\left(1 + 2 \frac{\|\mathbf{u} - \mathbf{v}\|^2}{(1 - \|\mathbf{u}\|^2)(1 - \|\mathbf{v}\|^2)} \right)$$

<ul class="text-xs text-slate-300 light:text-slate-700 space-y-2 list-disc list-inside leading-relaxed">

Geometric Model: Continuous representation of tree-like hierarchical manifolds with constant negative curvature.
Boundary Behavior: Distance approaches infinity as points approach the disk unit boundary $\|\mathbf{u}\| \to 1$.

<!-- Jaccard & Fuzzy Simplicial Sets Card -->

UMAP Topology
<h3 class="text-lg font-bold text-white light:text-slate-900">Fuzzy Simplicial Set Probabilities</h3>

$p_{i|j}$

$$p_{i|j} = \exp\left(-\frac{\max(0, d(x_i, x_j) - \rho_i)}{\sigma_i}\right)$$

<ul class="text-xs text-slate-300 light:text-slate-700 space-y-2 list-disc list-inside leading-relaxed">
Local Riemannian Metric: Adapts local distance scales (σ_i) based on the nearest neighbor distance (ρ_i).
Fuzzy Set Union: Combines directed graph edges via $p_{ij} = p_{i|j} + p_{j|i} - p_{i|j}p_{j|i}$.

</section>

<!-- ================ TAB 5: MATHEMATICAL GLOSSARY ================ -->
<section id="tab-glossary" class="hidden space-y-6">

```
            <!-- Search & Filters -->
            <div class="flex flex-col sm:flex-row items-center justify-between gap-4 pb-4 border-b
border-slate-800 light:border-slate-200">
                <div>
                    <h2 class="text-lg font-bold text-white light:text-slate-900">Comprehensive
Glossary</h2>
                    <p class="text-xs text-slate-400 light:text-slate-600">Metric spaces, manifold
embedding terminology, and non-Euclidean geometry</p>
                </div>

                <div class="w-full sm:w-80 space-y-2">
                    <div class="relative">
                        <i class="fa-solid fa-magnifying-glass absolute left-3 top-3 text-slate-500
text-xs"></i>
                        <input type="text" id="glossarySearch" oninput="filterGlossary()"
placeholder="Search terms, formulas..." class="w-full bg-slate-900 light:bg-white border
border-slate-700 light:border-slate-300 rounded-xl pl-9 pr-4 py-2 text-xs text-slate-200
light:text-slate-800 focus:ring-2 focus:ring-brand-500 outline-none">
                    </div>
                    <div class="flex flex-wrap gap-1 text-[10px]" id="glossaryCategoryFilters">
                        <button onclick="setGlossaryCategory('All')" class="px-2 py-0.5 rounded
bg-brand-600 text-white font-medium filter-btn">All</button>
                        <button onclick="setGlossaryCategory('High-Dim')" class="px-2 py-0.5
rounded bg-slate-800 hover:bg-slate-700 text-slate-300 filter-btn">High-Dim</button>
                        <button onclick="setGlossaryCategory('UMAP')" class="px-2 py-0.5 rounded
bg-slate-800 hover:bg-slate-700 text-slate-300 filter-btn">UMAP</button>
                        <button onclick="setGlossaryCategory('Hyperbolic')" class="px-2 py-0.5
rounded bg-slate-800 hover:bg-slate-700 text-slate-300 filter-btn">Hyperbolic</button>
                        <button onclick="setGlossaryCategory('Metrics')" class="px-2 py-0.5 rounded
bg-slate-800 hover:bg-slate-700 text-slate-300 filter-btn">Metrics</button>
                    </div>
                </div>
            </div>

            <!-- Glossary Terms Container -->
            <div id="glossaryList" class="grid grid-cols-1 md:grid-cols-2 gap-4">
                <!-- Term items inserted via JavaScript -->
            </div>
        </div>
    </section>

</main>

<!-- Footer -->
```

```
  <footer class="border-t border-slate-800 light:border-slate-200 py-4 text-center text-xs text-slate-500 light:text-slate-600">
    <p>Interactive High-Dimensional Spatial Relationships & UMAP Geometry Engine • Single-File Application</p>
  </footer>

  <!-- JavaScript Application Logic -->
  <script>
    // ================ APPLICATION STATE ================
    const state = {
      activeTab: 'calculator',
      vectorA: new Float64Array([0.25, -0.74, 0.81, 0.12, 0.55, -0.32]),
      vectorB: new Float64Array([0.18, -0.62, 0.95, 0.04, 0.41, -0.10]),
      calcResult: null,
      graph: {
        axisX: 0,
        axisY: 1,
        axisZ: 2,
        collapseMode: 'l2norm',
        radialMode: 'dim4',
        // 3D Camera Orbit State
        rotX: 0.4,
        rotY: -0.6,
        zoom: 1.0,
        isDragging: false,
        lastMouseX: 0,
        lastMouseY: 0,
        u3d: { x: 0, y: 0, z: 0 },
        v3d: { x: 0, y: 0, z: 0 }
      },
      umap: {
        points: [],
        edges: [],
        inputMetric: 'cosine',
        outputMetric: 'hyperboloid',
        nNeighbors: 15,
        curvature: 1.0
      },
      glossaryCategory: 'All'
    };

    // ================ TAB NAVIGATION ================
    function switchTab(tabId) {
      state.activeTab = tabId;
```

```
        // Hide all tab sections
        const tabs = ['calculator', 'graph', 'umap', 'formulas', 'glossary'];
        tabs.forEach(t => {
            const sec = document.getElementById(`tab-${t}`);
            if (sec) sec.classList.add('hidden');

            // Desktop nav tabs
            const navBtn = document.getElementById(`nav-${t}`);
            if (navBtn) {
                if (t === tabId) {
                    navBtn.classList.add('active', 'text-slate-300');
                    navBtn.classList.remove('text-slate-400');
                } else {
                    navBtn.classList.remove('active', 'text-slate-300');
                    navBtn.classList.add('text-slate-400');
                }
            }

            // Mobile nav tabs
            const mobNavBtn = document.getElementById(`mob-nav-${t}`);
            if (mobNavBtn) {
                if (t === tabId) {
                    mobNavBtn.className = "px-3 py-1.5 rounded-md bg-brand-600 text-white
font-medium whitespace-nowrap";
                } else {
                    mobNavBtn.className = "px-3 py-1.5 rounded-md text-slate-400
hover:bg-slate-800 whitespace-nowrap";
                }
            }
        });

        // Show active section
        const activeSec = document.getElementById(`tab-${tabId}`);
        if (activeSec) activeSec.classList.remove('hidden');

        // Trigger canvas redraws or calculations when switching tabs
        if (tabId === 'graph') {
            resizeGraphCanvas();
            syncGraphFromCalculator();
        } else if (tabId === 'umap') {
            resizeCanvas();
            if (state.umap.points.length === 0) setUmapPreset('hierarchical');
            else renderUmap();
```

```
        } else if (tabId === 'glossary') {
            renderGlossary();
        }

        if (window.MathJax && window.MathJax.typesetPromise) {
            window.MathJax.typesetPromise();
        }
    }

    // ================= EXPANDED GLOSSARY DATA =================
    const GLOSSARY_ITEMS = [
        { term: "Curse of Dimensionality", category: "High-Dim", desc: "As dimensionality N grows, space volume increases exponentially, causing distance metrics to lose contrast.", latex: "\\lim_{N \\to \\infty} \\frac{D_{\\max} - D_{\\min}}{D_{\\min}} = 0" },
        { term: "Poincaré Disk Model", category: "Hyperbolic", desc: "A conformal model of hyperbolic geometry mapped into the unit disk where geodesics form circular arcs orthogonal to the boundary.", latex: "d_{\\mathcal{H}}(u,v) = \\text{arcosh}\\left(1 + 2\\frac{\\|u-v\\|^2}{(1-\\|u\\|^2)(1-\\|v\\|^2)}\\right)" },
        { term: "Cosine Similarity", category: "Metrics", desc: "Measures orientation similarity between two vectors regardless of magnitude.", latex: "\\text{Sim}_{cos}(u,v) = \\frac{u \\cdot v}{\\|u\\|_2 \\|v\\|_2}" },
        { term: "Manhattan (Taxicab) Distance", category: "Metrics", desc: "Distance calculated along axis-aligned grid paths, often robust in high dimensions.", latex: "d_1(u,v) = \\sum |u_i - v_i|" },
        { term: "Mahalanobis Distance", category: "Metrics", desc: "Distance metric scaled according to feature covariances and variance.", latex: "d_M(u,v) = \\sqrt{(u-v)^T \\Sigma^{-1} (u-v)}" },
        { term: "Fuzzy Simplicial Set", category: "UMAP", desc: "Topological structure representing high-dimensional manifolds as weighted metric fuzzy graphs.", latex: "p_{i|j} = \\exp\\left(-\\frac{\\max(0, d(x_i, x_j) - \\rho_i)}{\\sigma_i}\\right)" }
    ];

    // ================= WINDOW INIT ON LOAD =================
    window.addEventListener('load', () => {
        // Set default vector inputs if empty
        if (!document.getElementById('vectorAInput').value) {
            document.getElementById('vectorAInput').value = Array.from(state.vectorA).join(', ');
        }
        if (!document.getElementById('vectorBInput').value) {
            document.getElementById('vectorBInput').value = Array.from(state.vectorB).join(', ');
        }

        // Init canvas engines
        initGraphCanvas();
```

```
        initUmapCanvas();
        renderGlossary();
        runCalculation();

        // Handle browser resize
        window.addEventListener('resize', () => {
            if (state.activeTab === 'graph') resizeGraphCanvas();
            if (state.activeTab === 'umap') resizeCanvas();
        });
    });

    // ================= 3D GRAPH CANVAS INTERACTIVITY ================
    let graphCanvas, graphCtx;

    function initGraphCanvas() {
        graphCanvas = document.getElementById('graphCanvas');
        if (!graphCanvas) return;
        graphCtx = graphCanvas.getContext('2d');

        // 3D Orbiting Drag Events
        graphCanvas.addEventListener('mousedown', (e) => {
            state.graph.isDragging = true;
            state.graph.lastMouseX = e.clientX;
            state.graph.lastMouseY = e.clientY;
        });

        window.addEventListener('mousemove', (e) => {
            if (!state.graph.isDragging) return;
            const dx = e.clientX - state.graph.lastMouseX;
            const dy = e.clientY - state.graph.lastMouseY;
            state.graph.rotY += dx * 0.008;
            state.graph.rotX += dy * 0.008;
            // Clamp elevation to avoid gimbal flip
            state.graph.rotX = Math.max(-Math.PI / 2 + 0.1, Math.min(Math.PI / 2 - 0.1,
state.graph.rotX));
            state.graph.lastMouseX = e.clientX;
            state.graph.lastMouseY = e.clientY;
            renderGraph();
        });

        window.addEventListener('mouseup', () => {
            state.graph.isDragging = false;
        });
```

```
    // Touch support for 3D Orbit
    graphCanvas.addEventListener('touchstart', (e) => {
        if (e.touches.length > 0) {
            state.graph.isDragging = true;
            state.graph.lastMouseX = e.touches[0].clientX;
            state.graph.lastMouseY = e.touches[0].clientY;
        }
    }, { passive: true });

    graphCanvas.addEventListener('touchmove', (e) => {
        if (!state.graph.isDragging || e.touches.length === 0) return;
        const dx = e.touches[0].clientX - state.graph.lastMouseX;
        const dy = e.touches[0].clientY - state.graph.lastMouseY;
        state.graph.rotY += dx * 0.008;
        state.graph.rotX += dy * 0.008;
        state.graph.lastMouseX = e.touches[0].clientX;
        state.graph.lastMouseY = e.touches[0].clientY;
        renderGraph();
    }, { passive: true });

    window.addEventListener('touchend', () => { state.graph.isDragging = false; });

    resizeGraphCanvas();
}

function reset3DCamera() {
    state.graph.rotX = 0.4;
    state.graph.rotY = -0.6;
    renderGraph();
}

function resizeGraphCanvas() {
    if (!graphCanvas) return;
    const rect = graphCanvas.parentElement.getBoundingClientRect();
    graphCanvas.width = rect.width * window.devicePixelRatio;
    graphCanvas.height = rect.height * window.devicePixelRatio;
    renderGraph();
}

// ================= SYNC CALCULATOR WITH 3D GRAPH =================
function syncGraphFromCalculator() {
    try {
        const inputA = document.getElementById('vectorAInput');
        const inputB = document.getElementById('vectorBInput');
```

```
        if (!inputA || !inputB) return;

        state.vectorA = parseVector(inputA.value);
        state.vectorB = parseVector(inputB.value);

        const N = state.vectorA.length;
        const badge = document.getElementById('highDimBadge');
        if (badge) badge.textContent = `Dim: ${N}D`;

        // Populate 3D Dimension Selectors
        const selectX = document.getElementById('selectXDim');
        const selectY = document.getElementById('selectYDim');
        const selectZ = document.getElementById('selectZDim');

        if (selectX && selectY && selectZ) {
            selectX.innerHTML = '';
            selectY.innerHTML = '';
            selectZ.innerHTML = '';

            for (let i = 0; i < N; i++) {
                const label = `Dimension ${i + 1} (u: ${state.vectorA[i].toFixed(2)}, v:
${state.vectorB[i].toFixed(2)})`;

                const optX = document.createElement('option');
                optX.value = i; optX.textContent = label;
                if (i === 0) optX.selected = true;
                selectX.appendChild(optX);

                const optY = document.createElement('option');
                optY.value = i; optY.textContent = label;
                if (i === (N > 1 ? 1 : 0)) optY.selected = true;
                selectY.appendChild(optY);

                const optZ = document.createElement('option');
                optZ.value = i; optZ.textContent = label;
                if (i === (N > 2 ? 2 : (N > 1 ? 1 : 0))) optZ.selected = true;
                selectZ.appendChild(optZ);
            }
        }

        state.graph.axisX = 0;
        state.graph.axisY = N > 1 ? 1 : 0;
        state.graph.axisZ = N > 2 ? 2 : (N > 1 ? 1 : 0);
```

```
        updateDimensionMapping();
    } catch (err) {
        console.warn("Sync graph error:", err);
    }
}

function updateDimensionMapping() {
    const selectX = document.getElementById('selectXDim');
    const selectY = document.getElementById('selectYDim');
    const selectZ = document.getElementById('selectZDim');
    const selectCollapse = document.getElementById('selectCollapseMode');
    const selectRadial = document.getElementById('selectRadialMode');

    if (selectX) state.graph.axisX = parseInt(selectX.value) || 0;
    if (selectY) state.graph.axisY = parseInt(selectY.value) || 0;
    if (selectZ) state.graph.axisZ = parseInt(selectZ.value) || 0;
    if (selectCollapse) state.graph.collapseMode = selectCollapse.value;
    if (selectRadial) state.graph.radialMode = selectRadial.value;

    // Project Vector A and B into 3D space
    state.graph.u3d = projectVectorTo3D(state.vectorA);
    state.graph.v3d = projectVectorTo3D(state.vectorB);

    updateInputFieldsFrom3D();
    renderGraph();
}

// Handles manual input changes from inputUx, inputUy, inputVx, inputVy
function updateGraphFromInputs() {
    const inputUx = document.getElementById('inputUx');
    const inputUy = document.getElementById('inputUy');
    const inputVx = document.getElementById('inputVx');
    const inputVy = document.getElementById('inputVy');

    if (!inputUx || !inputUy || !inputVx || !inputVy) return;

    const ux = parseFloat(inputUx.value) || 0;
    const uy = parseFloat(inputUy.value) || 0;
    const vx = parseFloat(inputVx.value) || 0;
    const vy = parseFloat(inputVy.value) || 0;

    const N = state.vectorA.length;
    if (N > 0) {
        state.vectorA[state.graph.axisX % N] = ux;
```

```
        state.vectorA[state.graph.axisY % N] = uy;
    }
    if (N > 0) {
        state.vectorB[state.graph.axisX % N] = vx;
        state.vectorB[state.graph.axisY % N] = vy;
    }

    // Sync back to textareas in Tab 1
    const inputA = document.getElementById('vectorAInput');
    const inputB = document.getElementById('vectorBInput');
    if (inputA) inputA.value = Array.from(state.vectorA).map(x => x.toFixed(4)).join(', ');
    if (inputB) inputB.value = Array.from(state.vectorB).map(x => x.toFixed(4)).join(', ');

    // Recalculate 3D projections and overall metrics
    state.graph.u3d = projectVectorTo3D(state.vectorA);
    state.graph.v3d = projectVectorTo3D(state.vectorB);

    runCalculation();
    renderGraph();
}

function updateInputFieldsFrom3D() {
    const inputUx = document.getElementById('inputUx');
    const inputUy = document.getElementById('inputUy');
    const inputVx = document.getElementById('inputVx');
    const inputVy = document.getElementById('inputVy');

    if (!inputUx || !inputUy || !inputVx || !inputVy) return;

    const N = state.vectorA.length;
    if (N > 0) {
        inputUx.value = state.vectorA[state.graph.axisX % N].toFixed(2);
        inputUy.value = state.vectorA[state.graph.axisY % N].toFixed(2);
        inputVx.value = state.vectorB[state.graph.axisX % N].toFixed(2);
        inputVy.value = state.vectorB[state.graph.axisY % N].toFixed(2);
    }
}

// Projects high-dimensional vectors to 3D basis coordinates
function projectVectorTo3D(vec) {
    const N = vec.length;
    if (N === 0) return { x: 0, y: 0, z: 0 };

    let x = vec[state.graph.axisX % N] || 0;
```

```
let y = vec[state.graph.axisY % N] || 0;
let z = vec[state.graph.axisZ % N] || 0;

if (state.graph.collapseMode === 'l2norm') {
    let sumSq = 0;
    for (let i = 0; i < N; i++) {
        if (i !== state.graph.axisX % N && i !== state.graph.axisY % N) {
            sumSq += vec[i] * vec[i];
        }
    }
    z = Math.sqrt(sumSq);
} else if (state.graph.collapseMode === 'mean') {
    let sum = 0, count = 0;
    for (let i = 0; i < N; i++) {
        if (i !== state.graph.axisX % N && i !== state.graph.axisY % N) {
            sum += vec[i];
            count++;
        }
    }
    z = count > 0 ? sum / count : 0;
} else if (state.graph.collapseMode === 'sum') {
    let sum = 0;
    for (let i = 0; i < N; i++) {
        if (i !== state.graph.axisX % N && i !== state.graph.axisY % N) {
            sum += vec[i];
        }
    }
    z = sum;
}

if (state.graph.radialMode === 'dim4' && N >= 4) {
    const r = Math.abs(vec[3]);
    const currentRadius = Math.hypot(x, y, z) || 1;
    x = (x / currentRadius) * r;
    y = (y / currentRadius) * r;
    z = (z / currentRadius) * r;
} else if (state.graph.radialMode === 'fullNorm') {
    let normSq = 0;
    for (let i = 0; i < N; i++) normSq += vec[i] * vec[i];
    const r = Math.sqrt(normSq);
    const currentRadius = Math.hypot(x, y, z) || 1;
    x = (x / currentRadius) * r;
    y = (y / currentRadius) * r;
    z = (z / currentRadius) * r;
```

```
    }

    return { x, y, z };
}

// 3D Point Projection to 2D Canvas Viewport (Standard Right-Handed Camera Matrix)
function project3Dto2D(point3d, center, scale) {
    const cosY = Math.cos(state.graph.rotY);
    const sinY = Math.sin(state.graph.rotY);
    const cosX = Math.cos(state.graph.rotX);
    const sinX = Math.sin(state.graph.rotX);

    // Yaw (Rotation around Y-axis)
    const x1 = point3d.x * cosY + point3d.z * sinY;
    const y1 = point3d.y;
    const z1 = -point3d.x * sinY + point3d.z * cosY;

    // Pitch (Rotation around X-axis)
    const x2 = x1;
    const y2 = y1 * cosX - z1 * sinX;
    const z2 = y1 * sinX + z1 * cosX;

    return {
        x: center.x + x2 * scale,
        y: center.y - y2 * scale, // Invert Y for canvas coordinate system
        z: z2 // Depth for sorting
    };
}

function renderGraph() {
    if (!graphCtx || !graphCanvas) return;
    const w = graphCanvas.width;
    const h = graphCanvas.height;
    const center = { x: w / 2, y: h / 2 };
    const scale = (Math.min(w, h) / 8) * state.graph.zoom;
    const isDark = document.documentElement.classList.contains('dark');

    graphCtx.clearRect(0, 0, w, h);

    // Origin
    const origin2D = project3Dto2D({ x: 0, y: 0, z: 0 }, center, scale);

    // Draw 3D Axes (X: Red, Y: Green, Z: Blue/Purple)
    const axisLength = 3;
```

```
        const axisX2D = project3Dto2D({ x: axisLength, y: 0, z: 0 }, center, scale);
        const axisY2D = project3Dto2D({ x: 0, y: axisLength, z: 0 }, center, scale);
        const axisZ2D = project3Dto2D({ x: 0, y: 0, z: axisLength }, center, scale);

        // Floor Grid Plane (XZ plane at y=0)
        graphCtx.strokeStyle = isDark ? 'rgba(255, 255, 255, 0.08)' : 'rgba(0, 0, 0, 0.08)';
        graphCtx.lineWidth = 1 * window.devicePixelRatio;
        for (let i = -3; i <= 3; i++) {
            const p1 = project3Dto2D({ x: i, y: 0, z: -3 }, center, scale);
            const p2 = project3Dto2D({ x: i, y: 0, z: 3 }, center, scale);
            graphCtx.beginPath(); graphCtx.moveTo(p1.x, p1.y); graphCtx.lineTo(p2.x, p2.y);
graphCtx.stroke();

            const p3 = project3Dto2D({ x: -3, y: 0, z: i }, center, scale);
            const p4 = project3Dto2D({ x: 3, y: 0, z: i }, center, scale);
            graphCtx.beginPath(); graphCtx.moveTo(p3.x, p3.y); graphCtx.lineTo(p4.x, p4.y);
graphCtx.stroke();
        }

        // Draw Axes Lines
        draw3DLine(graphCtx, origin2D, axisX2D, '#ef4444', 2, 'X');
        draw3DLine(graphCtx, origin2D, axisY2D, '#10b981', 2, 'Y');
        draw3DLine(graphCtx, origin2D, axisZ2D, '#a855f7', 2, 'Z');

        const u3d = state.graph.u3d;
        const v3d = state.graph.v3d;

        const u2d = project3Dto2D(u3d, center, scale);
        const v2d = project3Dto2D(v3d, center, scale);

        // Bounding Wireframe Box
        const chkBox = document.getElementById('chkBoundingBox');
        if (chkBox && chkBox.checked) {
            drawWireBox(graphCtx, u3d, center, scale, 'rgba(59, 130, 246, 0.15)');
            drawWireBox(graphCtx, v3d, center, scale, 'rgba(16, 185, 129, 0.15)');
        }

        // Drop lines to XZ floor plane (y=0)
        const chkProj = document.getElementById('chkProjection');
        if (chkProj && chkProj.checked) {
            const uFloor = project3Dto2D({ x: u3d.x, y: 0, z: u3d.z }, center, scale);
            const vFloor = project3Dto2D({ x: v3d.x, y: 0, z: v3d.z }, center, scale);

            drawDashedLine(graphCtx, u2d, uFloor, 'rgba(59, 130, 246, 0.5)');
```

```
    drawDashedLine(graphCtx, v2d, vFloor, 'rgba(16, 185, 129, 0.5)');
}

// 3D Taxicab Path
const chkMan = document.getElementById('chkManhattan');
if (chkMan && chkMan.checked) {
    const pStep1 = project3Dto2D({ x: v3d.x, y: u3d.y, z: u3d.z }, center, scale);
    const pStep2 = project3Dto2D({ x: v3d.x, y: v3d.y, z: u3d.z }, center, scale);

    drawDashedLine(graphCtx, u2d, pStep1, '#a855f7');
    drawDashedLine(graphCtx, pStep1, pStep2, '#a855f7');
    drawDashedLine(graphCtx, pStep2, v2d, '#a855f7');
}

// Vector Projection onto Vector A
const chkVecProj = document.getElementById('chkVectorProj');
const normU2 = u3d.x * u3d.x + u3d.y * u3d.y + u3d.z * u3d.z;
if (chkVecProj && chkVecProj.checked && normU2 > 1e-6) {
    const dot = u3d.x * v3d.x + u3d.y * v3d.y + u3d.z * v3d.z;
    const scalarProj = dot / normU2;
    const projPoint3d = {
        x: u3d.x * scalarProj,
        y: u3d.y * scalarProj,
        z: u3d.z * scalarProj
    };
    const proj2d = project3Dto2D(projPoint3d, center, scale);
    drawDashedLine(graphCtx, v2d, proj2d, '#10b981');
    draw3DVectorNode(graphCtx, origin2D, proj2d, '#34d399', '');
}

// 3D Angle Arc
const chkAngle = document.getElementById('chkAngleArc');
const normU = Math.sqrt(normU2);
const normV = Math.sqrt(v3d.x * v3d.x + v3d.y * v3d.y + v3d.z * v3d.z);
if (chkAngle && chkAngle.checked && normU > 1e-6 && normV > 1e-6) {
    const steps = 20;
    graphCtx.beginPath();
    for (let i = 0; i <= steps; i++) {
        const t = i / steps;
        const interpPoint = {
            x: (1 - t) * u3d.x / normU + t * v3d.x / normV,
            y: (1 - t) * u3d.y / normU + t * v3d.y / normV,
            z: (1 - t) * u3d.z / normU + t * v3d.z / normV
        };
```

```
            const normI = Math.sqrt(interpPoint.x ** 2 + interpPoint.y ** 2 + interpPoint.z ** 2) ||
1;
            const arc3d = {
                x: (interpPoint.x / normI) * 0.8,
                y: (interpPoint.y / normI) * 0.8,
                z: (interpPoint.z / normI) * 0.8
            };
            const arc2d = project3Dto2D(arc3d, center, scale);
            if (i === 0) graphCtx.moveTo(arc2d.x, arc2d.y);
            else graphCtx.lineTo(arc2d.x, arc2d.y);
        }
        graphCtx.strokeStyle = '#f59e0b';
        graphCtx.lineWidth = 2 * window.devicePixelRatio;
        graphCtx.stroke();
    }

    // 3D Euclidean Distance Line
    const chkEuc = document.getElementById('chkEuclidean');
    if (chkEuc && chkEuc.checked) {
        graphCtx.beginPath();
        graphCtx.moveTo(u2d.x, u2d.y);
        graphCtx.lineTo(v2d.x, v2d.y);
        graphCtx.strokeStyle = '#6366f1';
        graphCtx.lineWidth = 2.5 * window.devicePixelRatio;
        graphCtx.stroke();
    }

    // Vector A Arrow in 3D
    draw3DVectorNode(graphCtx, origin2D, u2d, '#3b82f6', 'Vector A');
    // Vector B Arrow in 3D
    draw3DVectorNode(graphCtx, origin2D, v2d, '#10b981', 'Vector B');

    // Calculate 3D Projected Distance Values
    const dx = u3d.x - v3d.x;
    const dy = u3d.y - v3d.y;
    const dz = u3d.z - v3d.z;

    const euclid3D = Math.sqrt(dx * dx + dy * dy + dz * dz);
    const manhattan3D = Math.abs(dx) + Math.abs(dy) + Math.abs(dz);

    const dot3D = u3d.x * v3d.x + u3d.y * v3d.y + u3d.z * v3d.z;
    const cos3D = (normU && normV) ? Math.max(-1, Math.min(1, dot3D / (normU *
normV))) : 0;
    const angle3D = Math.acos(cos3D) * (180 / Math.PI);
```

```
        const coordA = document.getElementById('graphCoordA');
        const coordB = document.getElementById('graphCoordB');
        const valEuc = document.getElementById('graphValEuclid');
        const valMan = document.getElementById('graphValManhattan');
        const valAng = document.getElementById('graphValAngle');

        if (coordA) coordA.textContent = `[${u3d.x.toFixed(2)}, ${u3d.y.toFixed(2)},
${u3d.z.toFixed(2)}]`;
        if (coordB) coordB.textContent = `[${v3d.x.toFixed(2)}, ${v3d.y.toFixed(2)},
${v3d.z.toFixed(2)}]`;
        if (valEuc) valEuc.textContent = euclid3D.toFixed(3);
        if (valMan) valMan.textContent = manhattan3D.toFixed(3);
        if (valAng) valAng.textContent = `${angle3D.toFixed(1)}°`;
    }

    function draw3DLine(ctx, p1, p2, color, width, label) {
        ctx.beginPath();
        ctx.moveTo(p1.x, p1.y);
        ctx.lineTo(p2.x, p2.y);
        ctx.strokeStyle = color;
        ctx.lineWidth = width * window.devicePixelRatio;
        ctx.stroke();

        if (label) {
            ctx.fillStyle = color;
            ctx.font = `${10 * window.devicePixelRatio}px JetBrains Mono`;
            ctx.fillText(label, p2.x + 5, p2.y + 5);
        }
    }

    function drawDashedLine(ctx, p1, p2, color) {
        ctx.beginPath();
        ctx.moveTo(p1.x, p1.y);
        ctx.lineTo(p2.x, p2.y);
        ctx.strokeStyle = color;
        ctx.lineWidth = 1.5 * window.devicePixelRatio;
        ctx.setLineDash([4, 4]);
        ctx.stroke();
        ctx.setLineDash([]);
    }

    function drawWireBox(ctx, point3d, center, scale, color) {
        const corners = [
```

```
        { x: 0, y: 0, z: 0 }, { x: point3d.x, y: 0, z: 0 },
        { x: point3d.x, y: point3d.y, z: 0 }, { x: 0, y: point3d.y, z: 0 },
        { x: 0, y: 0, z: point3d.z }, { x: point3d.x, y: 0, z: point3d.z },
        { x: point3d.x, y: point3d.y, z: point3d.z }, { x: 0, y: point3d.y, z: point3d.z }
    ].map(p => project3Dto2D(p, center, scale));

    ctx.strokeStyle = color;
    ctx.lineWidth = 1 * window.devicePixelRatio;

    const edges = [
        [0, 1], [1, 2], [2, 3], [3, 0],
        [4, 5], [5, 6], [6, 7], [7, 4],
        [0, 4], [1, 5], [2, 6], [3, 7]
    ];

    edges.forEach(([i, j]) => {
        ctx.beginPath();
        ctx.moveTo(corners[i].x, corners[i].y);
        ctx.lineTo(corners[j].x, corners[j].y);
        ctx.stroke();
    });
}

function draw3DVectorNode(ctx, origin, target, color, label) {
    ctx.beginPath();
    ctx.moveTo(origin.x, origin.y);
    ctx.lineTo(target.x, target.y);
    ctx.strokeStyle = color;
    ctx.lineWidth = 3 * window.devicePixelRatio;
    ctx.stroke();

    // Head Sphere Node
    ctx.beginPath();
    ctx.arc(target.x, target.y, 7 * window.devicePixelRatio, 0, Math.PI * 2);
    ctx.fillStyle = color;
    ctx.fill();
    ctx.strokeStyle = '#ffffff';
    ctx.lineWidth = 2 * window.devicePixelRatio;
    ctx.stroke();
}

// ================= GLOSSARY CATEGORY FILTERING =================
function setGlossaryCategory(cat) {
    state.glossaryCategory = cat;
```

```
        const btns = document.querySelectorAll('#glossaryCategoryFilters button');
        btns.forEach(b => {
            if (b.textContent === cat) {
                b.className = "px-2 py-0.5 rounded bg-brand-600 text-white font-medium
filter-btn";
            } else {
                b.className = "px-2 py-0.5 rounded bg-slate-800 hover:bg-slate-700 text-slate-300
filter-btn";
            }
        });
        filterGlossary();
    }

    function renderGlossary() {
        const container = document.getElementById('glossaryList');
        if (!container) return;
        container.innerHTML = GLOSSARY_ITEMS.map((item, idx) => `
            <div class="glass-panel p-4 rounded-xl border space-y-2">
                <div class="flex justify-between items-start">
                    <h3 class="font-bold text-sm text-white light:text-slate-900">${item.term}</h3>
                    <span class="text-[10px] font-mono px-2 py-0.5 rounded bg-brand-500/10
text-brand-400 border border-brand-500/20">${item.category}</span>
                </div>
                <p class="text-xs text-slate-300 light:text-slate-700
leading-relaxed">${item.desc}</p>
                <div class="bg-slate-900/90 light:bg-slate-100 p-2.5 rounded-lg font-mono
text-center text-xs text-brand-300 light:text-brand-700 overflow-x-auto">
                    $item.latex
                </div>
            </div>
        `).join('');
        filterGlossary();
    }

    function filterGlossary() {
        const searchInput = document.getElementById('glossarySearch');
        const query = (searchInput ? searchInput.value : '').toLowerCase();
        const list = document.getElementById('glossaryList');
        if (!list) return;

        const cards = list.children;
        GLOSSARY_ITEMS.forEach((item, idx) => {
            const matchCategory = state.glossaryCategory === 'All' || item.category ===
state.glossaryCategory;
```

```
            const matchQuery = item.term.toLowerCase().includes(query) ||
                               item.desc.toLowerCase().includes(query) ||
                               item.category.toLowerCase().includes(query);

            if (cards[idx]) {
                cards[idx].style.display = (matchCategory && matchQuery) ? 'block' : 'none';
            }
        });
    }

    // ================= VECTOR CALCULATOR COMPUTATIONS
=================
    function parseVector(text) {
        const parts = text.trim().split(/[\s,]+/).filter(x => x !== "");
        const arr = new Float64Array(parts.length);
        for (let i = 0; i < parts.length; i++) {
            const val = parseFloat(parts[i]);
            if (isNaN(val)) throw new Error(`Invalid number at position ${i + 1}`);
            arr[i] = val;
        }
        return arr;
    }

    function runCalculation() {
        const errBanner = document.getElementById('calcError');
        if (errBanner) errBanner.classList.add('hidden');

        try {
            const inputA = document.getElementById('vectorAInput');
            const inputB = document.getElementById('vectorBInput');
            if (!inputA || !inputB) return;

            state.vectorA = parseVector(inputA.value);
            state.vectorB = parseVector(inputB.value);

            const badgeA = document.getElementById('dimBadgeA');
            const badgeB = document.getElementById('dimBadgeB');
            if (badgeA) badgeA.textContent = `Dim: ${state.vectorA.length}`;
            if (badgeB) badgeB.textContent = `Dim: ${state.vectorB.length}`;

            if (state.vectorA.length !== state.vectorB.length) {
                throw new Error(`Dimension Mismatch: Vector A (
state.vectorA.lengthD)vsVectorB({state.vectorB.length}D)`);
            }
```

```
        const N = state.vectorA.length;
        if (N === 0) throw new Error("Vectors cannot be empty.");

        // Single-pass compute loop
        let dot = 0, normA2 = 0, normB2 = 0, manhattan = 0, euclidean2 = 0;
        for (let i = 0; i < N; i++) {
            const a = state.vectorA[i];
            const b = state.vectorB[i];
            const diff = a - b;

            dot += a * b;
            normA2 += a * a;
            normB2 += b * b;
            manhattan += Math.abs(diff);
            euclidean2 += diff * diff;
        }

        const normA = Math.sqrt(normA2);
        const normB = Math.sqrt(normB2);
        const euclidean = Math.sqrt(euclidean2);

        const EPS = 1e-12;
        let cosSim = 0;
        if (normA > EPS && normB > EPS) {
            cosSim = Math.max(-1, Math.min(1, dot / (normA * normB)));
        }

        const cosDist = 1 - cosSim;
        const angleDeg = Math.acos(cosSim) * (180 / Math.PI);

        state.calcResult = { N, dot, normA, normB, euclidean, manhattan, cosSim, cosDist,
angleDeg };

        // Update UI Display
        const normElA = document.getElementById('normValA');
        const normElB = document.getElementById('normValB');
        if (normElA) normElA.textContent = normA.toFixed(4);
        if (normElB) normElB.textContent = normB.toFixed(4);

        const resSim = document.getElementById('resCosineSim');
        const resEuc = document.getElementById('resEuclidean');
        const resCosD = document.getElementById('resCosDist');
        const resMan = document.getElementById('resManhattan');
```

```
            const resDotEl = document.getElementById('resDot');
            const resAngEl = document.getElementById('resAngle');

            if (resSim) resSim.textContent = cosSim.toFixed(4);
            if (resEuc) resEuc.textContent = euclidean.toFixed(4);
            if (resCosD) resCosD.textContent = cosDist.toFixed(4);
            if (resMan) resMan.textContent = manhattan.toFixed(2);
            if (resDotEl) resDotEl.textContent = dot.toFixed(4);
            if (resAngEl) resAngEl.textContent = `${angleDeg.toFixed(1)}°`;

            // Update Visual Progress Bars
            const barCos = document.getElementById('barCosine');
            const barEuc = document.getElementById('barEuclidean');
            const barCosD = document.getElementById('barCosDist');

            if (barCos) barCos.style.width = `${Math.max(0, (cosSim + 1) / 2 * 100)}%`;
            if (barEuc) barEuc.style.width = `${Math.min(100, (euclidean / (Math.sqrt(N) || 1)) *
50)}%`;
            if (barCosD) barCosD.style.width = `${Math.min(100, (cosDist / 2) * 100)}%`;

            // Step-by-Step Equations Breakdown
            const sDot = document.getElementById('stepDot');
            const sNormA = document.getElementById('stepNormA');
            const sNormB = document.getElementById('stepNormB');
            const sCos = document.getElementById('stepCosineEq');
            const sEuc = document.getElementById('stepEuclidEq');


            if (sDot) sDot.innerHTML = `1. Dot Product (∑ u_i v_i) = <span class="text-brand-400
font-bold">${dot.toFixed(6)}</span>`;
            if (sNormA) sNormA.innerHTML = `2. $\|\mathbf{u}\|^2_2 = \sum u_i^2$ =
${normA2.toFixed(6)} → $\|\mathbf{u}\|_2 = <span class="text-brand-400
font-bold">${normA.toFixed(6)}</span>`;
            if (sNormB) sNormB.innerHTML = `3. $\|\mathbf{v}\|^2_2 = \sum v_i^2$ =
${normB2.toFixed(6)} → $\|\mathbf{v}\|_2 = <span class="text-emerald-400
font-bold">${normB.toFixed(6)}</span>`;
            if (sCos) sCos.innerHTML = `4. Cosine Sim = $\\frac{${dot.toFixed(4)}}{${(normA *
normB).toFixed(4)}}$ = <span class="text-yellow-400 font-bold">${cosSim.toFixed(6)}</span>`;
            if (sEuc) sEuc.innerHTML = `5. Euclidean = $\\sqrt{${euclidean2.toFixed(6)}}$ =
<span class="text-indigo-400 font-bold">${euclidean.toFixed(6)}</span>`;

            if (window.MathJax && window.MathJax.typesetPromise) {
                window.MathJax.typesetPromise();
            }
```

```
    } catch (err) {
        if (errBanner) {
            errBanner.classList.remove('hidden');
            const msg = document.getElementById('calcErrorMsg');
            if (msg) msg.textContent = err.message;
        }
    }
}

function generateRandomVectors(dim) {
    const vecA = new Float64Array(dim);
    const vecB = new Float64Array(dim);
    for (let i = 0; i < dim; i++) {
        vecA[i] = Number(((Math.random() - 0.5) * 2).toFixed(4));
        vecB[i] = Number(((Math.random() - 0.5) * 2).toFixed(4));
    }
    const inputA = document.getElementById('vectorAInput');
    const inputB = document.getElementById('vectorBInput');
    if (inputA) inputA.value = Array.from(vecA).join(', ');
    if (inputB) inputB.value = Array.from(vecB).join(', ');
    runCalculation();
}

function normalizeInputs() {
    try {
        runCalculation();
        const normA = state.calcResult.normA;
        const normB = state.calcResult.normB;
        const inputA = document.getElementById('vectorAInput');
        const inputB = document.getElementById('vectorBInput');

        if (normA > 0 && inputA) {
            for (let i = 0; i < state.vectorA.length; i++) state.vectorA[i] /= normA;
            inputA.value = Array.from(state.vectorA).map(x => x.toFixed(4)).join(', ');
        }
        if (normB > 0 && inputB) {
            for (let i = 0; i < state.vectorB.length; i++) state.vectorB[i] /= normB;
            inputB.value = Array.from(state.vectorB).map(x => x.toFixed(4)).join(', ');
        }
        runCalculation();
    } catch (err) {}
}

function copyResults() {
```

```
        if (!state.calcResult) return;
        const str = JSON.stringify(state.calcResult, null, 2);
        navigator.clipboard ? navigator.clipboard.writeText(str) :
document.execCommand('copy');
    }

    // ================= UMAP & HYPERBOLIC VISUALIZATION =================
    let umapCanvas, umapCtx;

    function initUmapCanvas() {
        umapCanvas = document.getElementById('umapCanvas');
        if (!umapCanvas) return;
        umapCtx = umapCanvas.getContext('2d');
        resizeCanvas();
    }

    function resizeCanvas() {
        if (!umapCanvas) return;
        const parent = umapCanvas.parentElement;
        if (!parent) return;
        const rect = parent.getBoundingClientRect();
        umapCanvas.width = rect.width * window.devicePixelRatio;
        umapCanvas.height = rect.height * window.devicePixelRatio;
        renderUmap();
    }

    function setUmapPreset(preset) {
        state.umap.points = [];
        const count = 40;
        if (preset === 'hierarchical') {
            for (let i = 0; i < count; i++) {
                const depth = (i % 4) + 1;
                const angle = (Math.floor(i / 4) / (count / 4)) * Math.PI * 2;
                const r = (depth / 4) * 0.75;
                state.umap.points.push({
                    x: Math.cos(angle) * r,
                    y: Math.sin(angle) * r,
                    cluster: depth
                });
            }
        } else {
            for (let c = 0; c < 3; c++) {
                const cx = (c - 1) * 0.45;
                const cy = (c % 2 === 0 ? 0.3 : -0.3);
```

```
                for (let i = 0; i < 15; i++) {
                    state.umap.points.push({
                        x: cx + (Math.random() - 0.5) * 0.25,
                        y: cy + (Math.random() - 0.5) * 0.25,
                        cluster: c + 1
                    });
                }
            }
        }
        updateUmapSimulation();
    }

    function updateUmapSimulation() {
        const inMetric = document.getElementById('umapInputMetric');
        const outMetric = document.getElementById('umapOutputMetric');
        const sliderN = document.getElementById('sliderNeighbors');
        const sliderK = document.getElementById('sliderCurvature');

        if (inMetric) state.umap.inputMetric = inMetric.value;
        if (outMetric) state.umap.outputMetric = outMetric.value;
        if (sliderN) state.umap.nNeighbors = parseInt(sliderN.value);
        if (sliderK) state.umap.curvature = parseFloat(sliderK.value);

        const valN = document.getElementById('valNeighbors');
        const valK = document.getElementById('valCurvature');
        if (valN) valN.textContent = state.umap.nNeighbors;
        if (valK) valK.textContent = state.umap.curvature.toFixed(1);

        // Generate Fuzzy Simplicial Edges
        state.umap.edges = [];
        const pts = state.umap.points;
        for (let i = 0; i < pts.length; i++) {
            for (let j = i + 1; j < pts.length; j++) {
                const d = Math.hypot(pts[i].x - pts[j].x, pts[i].y - pts[j].y);
                if (d < 0.45) {
                    state.umap.edges.push({ i, j, weight: 1 - d / 0.45 });
                }
            }
        }

        // Update Python Snippet
        const pyCode = `import umap\nimport numpy as np\n\nmapper = umap.UMAP(\n
n_neighbors=${state.umap.nNeighbors},\n    min_dist=0.1,\n
```

```
metric="${state.umap.inputMetric}",\n    output_metric="${state.umap.outputMetric}",\n
random_state=42\n)\nembedding = mapper.fit_transform(X_high_dim)`;
        const pySnippet = document.getElementById('pythonCodeSnippet');
        if (pySnippet) pySnippet.textContent = pyCode;

        renderUmap();
    }

    function renderUmap() {
        if (!umapCtx || !umapCanvas) return;
        const w = umapCanvas.width;
        const h = umapCanvas.height;
        const center = { x: w / 2, y: h / 2 };
        const radius = Math.min(w, h) * 0.42;
        const isDark = document.documentElement.classList.contains('dark');

        umapCtx.clearRect(0, 0, w, h);

        if (state.umap.outputMetric === 'hyperboloid') {
            // Draw Poincaré Boundary Circle
            umapCtx.beginPath();
            umapCtx.arc(center.x, center.y, radius, 0, Math.PI * 2);
            umapCtx.strokeStyle = isDark ? '#ec4899' : '#db2777';
            umapCtx.lineWidth = 2 * window.devicePixelRatio;
            umapCtx.fillStyle = isDark ? 'rgba(236, 72, 153, 0.03)' : 'rgba(219, 39, 119, 0.03)';
            umapCtx.fill();
            umapCtx.stroke();
        }

        // Draw Fuzzy Edges
        umapCtx.strokeStyle = isDark ? 'rgba(59, 130, 246, 0.25)' : 'rgba(37, 99, 235, 0.25)';
        state.umap.edges.forEach(e => {
            const p1 = state.umap.points[e.i];
            const p2 = state.umap.points[e.j];
            umapCtx.lineWidth = e.weight * 2 * window.devicePixelRatio;
            umapCtx.beginPath();
            umapCtx.moveTo(center.x + p1.x * radius, center.y + p1.y * radius);
            umapCtx.lineTo(center.x + p2.x * radius, center.y + p2.y * radius);
            umapCtx.stroke();
        });

        // Draw Nodes
        const colors = ['#3b82f6', '#10b981', '#f59e0b', '#ec4899'];
        state.umap.points.forEach(p => {
```

```
            umapCtx.beginPath();
            umapCtx.arc(center.x + p.x * radius, center.y + p.y * radius, 5 *
window.devicePixelRatio, 0, Math.PI * 2);
            umapCtx.fillStyle = colors[(p.cluster - 1) % colors.length];
            umapCtx.fill();
            umapCtx.strokeStyle = '#ffffff';
            umapCtx.lineWidth = 1.5 * window.devicePixelRatio;
            umapCtx.stroke();
        });
    }

    function copyPythonCode() {
        const pySnippet = document.getElementById('pythonCodeSnippet');
        if (!pySnippet) return;
        const txt = pySnippet.textContent;
        navigator.clipboard ? navigator.clipboard.writeText(txt) :
document.execCommand('copy');
    }

    // ================= THEME & UI TOGGLE =================
    function toggleTheme() {
        const html = document.documentElement;
        const icon = document.getElementById('themeIcon');
        if (html.classList.contains('dark')) {
            html.classList.remove('dark');
            html.classList.add('light');
            if (icon) icon.className = 'fa-solid fa-sun text-amber-500';
        } else {
            html.classList.remove('light');
            html.classList.add('dark');
            if (icon) icon.className = 'fa-solid fa-moon text-slate-300';
        }
        if (state.activeTab === 'graph') renderGraph();
        if (state.activeTab === 'umap') renderUmap();
    }
  </script>
</body>
</html>
```